ИТМО

Analysis & Development of Algorithms

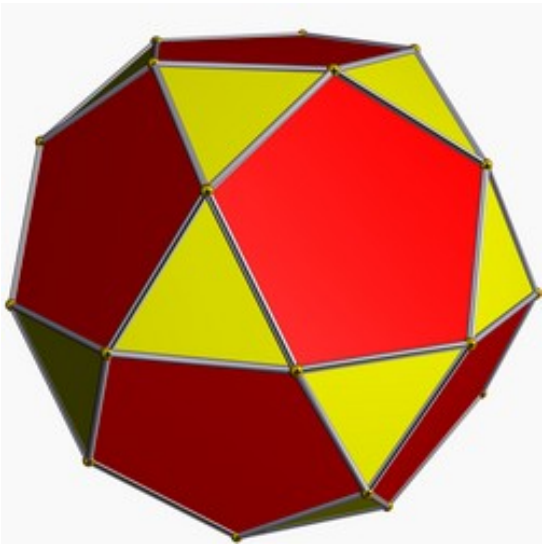
VIII

Anton Kovantsev
https://t.me/pop_antonij

Linear programming

Linear programming (LP), also called **linear optimization**, is a method to achieve the best outcome (such as maximum profit or lowest cost) in a mathematical model whose requirements and objective are represented by linear relationships.
A linear programming algorithm finds a point in the polytope where this function has the largest (or smallest) value if such a point exists.

Find a vector x
that maximizes $c^T x$
subject to $Ax \le b$; $x \ge 0$.
Dual problem,
minimize $b^T y$
subject to $A^T y = c$; $y \ge 0$.



3d convex polytope
(Icosidodecahedron)

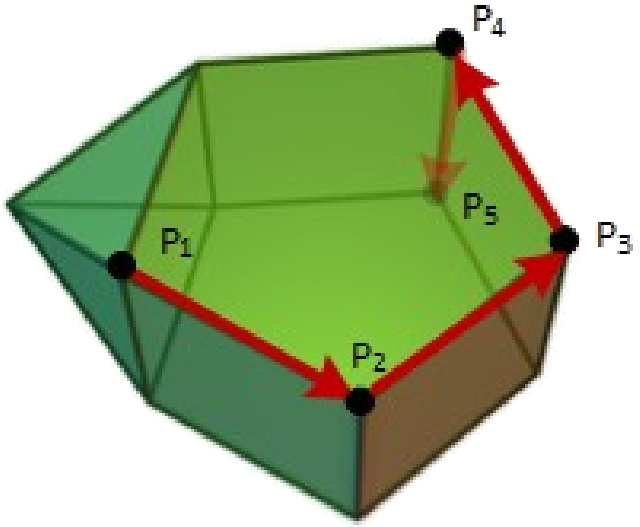
1930-s: Leonid Kantorovich (USSR) and Wassily Leontief (USA) independently delved into the practical applications of linear programming. Kantorovich focused on manufacturing schedules, while Leontief explored economic applications.
1949: George B. Dantzig "A Theorem on Linear Inequalities" => simplex algorithm.
1979: Leonid Khachiyan applied ellipsoid method (an iterative method for minimizing convex functions over convex sets).
1984: Narendra Karmarkar introduced a new interior-point method

Problem formulation: *Standard form* $c^T x \to \max;\ Ax \leqslant b;\ x \geqslant 0;\ b \geqslant 0$

Augmented form (slack form) $z \to \max;\ \begin{bmatrix} 1 & -c^T & 0 \\ 0 & A & I \end{bmatrix} \begin{bmatrix} z \\ x \\ s \end{bmatrix} = \begin{bmatrix} 0 \\ b \end{bmatrix}$

s – new variables that complement the old ones so that inequality becomes equality

Simplex algorithm



Polyhedron of simplex algorithm

Dantzig's simplex algorithm (or simplex method) performs iterations over the extreme points set until a condition called optimality criterion holds.
Based on these two theorems:

The feasible region for an LP problem is a convex set. Therefore, if an LP has an optimal solution, there must be an extreme point of the feasible region that is optimal

For an LP optimization problem, there is only one extreme point of the LP's feasible region regarding every basic feasible solution. Plus, there will be a minimum of one basic feasible solution corresponding to every extreme point in the feasible region.

These extreme poinst appears as the solution of the linear equations system.
Complexity: O(exp(N)) – the worst; polynomial on average.

Simplex algorithm

basis	x_1	x_2	...	x_n	
x_n	a_{11}	a_{12}	...	a_{1n}	b_1
x_{n-1}	a_{21}	a_{22}	...	a_{2n}	b_2
...	...	...	...	...	...
x_{n-2}	a_{m1}	a_{m2}	...	a_{mn}	b_m
Z	c_1	c_2	...	c_n	0

simplex tableau

Simplex tableau
Let there be a system of m equations and n unknowns (m < n).
We divide the variables into two sets:
(n-m) variables are set equal to zero, they are free (non-basic)
m variables are determined by the solution of the system of initial equations. Theyare called basic; and the corresponding resulting values of the variables are called the basic solution.
Pivoting the tableau (changing the basis) => move to the other vertex, new solution.

1) Entering variable:select a variable whose increase will lead to an increase in the functional.
If the task is for a minimum, we select the maximum positive element in the last row.
2) Leaving variable: The vector of the right parts is divided term by term by the column with the chosen coefficient
Among the obtained values, the minimum positive one is selected (negative and zero answers are not considered)
3) Next, the pivot row must be selected so that all the other basic variables remain positive. A calculation shows that this occurs when the resulting value of the entering variable is at a minimum.
4) Calculate new basic solution (Gaussian elimination)
5) Repeat until the optimality condition is satisfied
If the task is for the maximum, there are no negative coefficients in the functional line (i.e., with any change in the variables, the value of the final functional will not increase). If the task is for the minimum, there are no positive coefficients in the functional line (i.e., with any change in the variables, the value of the final functional will not decrease).

Simplex algorithm

1) The task formalization:

$$Z = c_1 x_1 + c_2 x_2 + \ldots + c_n x_n \rightarrow max$$

$$\begin{cases} a_{11} x_1 + \ldots + a_{1n} x_n \le b_1 \\ a_{21} x_1 + \ldots + a_{2n} x_n \le b_2 \\ \ldots \\ a_{m1} x_1 + \ldots + a_{mn} x_n \le b_m \end{cases}$$

2) Additional variables are introduced into the problem constraints:

$$y_j > 0 \; \forall \; j \in [1, m]$$

$$\begin{cases} a_{11} x_1 + \ldots + a_{1n} x_n + y_1 = b_1 \\ a_{21} x_1 + \ldots + a_{2n} x_n + y_2 = b_2 \\ \ldots \\ a_{m1} x_1 + \ldots + a_{mn} x_n + y_m = b_m \end{cases}$$

3) Tablleau:
First feasible solution:
(0, 0,…,0, b_1, b_2, …, b_m)

4) all $c_j \le 0$
YES — done.
NO — go ahead.

basis	**variables**							**b_i**
	x_1	x_2	...	x_n	y_1	...	y_m	
y_1	a_{11}	a_{12}	...	a_{1n}	1	0	0	b_1
y_2	a_{21}	a_{22}	...	a_{2n}	0	...	0	b_2
...	...	...	...	...	...	...	...	...
y_m	a_{m1}	a_{m2}	...	a_{mn}	0	0	1	b_m
c_j	**c_1**	**c_2**	**...**	**c_n**	**0**	**0**	**0**	Z

Simplex algorithm

5) Choose the column r (a variable entering the basis):

$$c_r = \max_{j \in [1, n+m]} \{ c_j \}$$

6) all $a_{ir} \le 0$.

YES — the objective function is unbounded and there is no solution,
NO — go ahead.

7) Choose the row s (a variable being removed from the basis) \:

$$D_s = \min_{i \in [1, m]} \left\{ \frac{b_i}{a_{ir}} \right\} \text{ для } a_{ir} > 0$$

8) Recalculation of simplex table elements (transition to a new basic solution)
For the elements of the pivot row:

$$a'_{sj} = \frac{a_{sj}}{a_{sr}} \quad j \in [1, n]; \quad b'_s = \frac{b_s}{a_{sr}}$$

For the elements of the pivot column, all elements except the pivot element must become equal to zero:

basis	**variables**							$\mathbf{b_i}$
	x_1	x_2	...	x_n	y_1	...	y_m	
y_1	a_{11}	0	...	a_{1n}	1	0	0	b_1
x_2	a_{21}/a_{22}	1	...	a_{2n}/a_{22}	$0/a_{22}$	...	$0/a_{22}$	b_2/a_{22}
...	...	...	...	...	...	...	...	...
y_m	a_{m1}	0	...	a_{mn}	0	0	1	b_m
$\mathbf{c_j}$	$\mathbf{c_1}$	**0**	**...**	$\mathbf{c_n}$	**0**	**0**	**0**	Z

Simplex algorithm

Алгоритм:

8) Elements not belonging to the pivot column or pivot row are recalculated using the so-called "rectangle rule": one mentally identifies a rectangle in which the element to be recalculated and the pivot element form one of the diagonals.

$$a'_{ij} = a_{ij} - \frac{a_{ir} a_{sj}}{a_{sr}}; \quad b'_i = b_i - \frac{a_{ir} b_s}{a_{sr}}; \quad c'_j = c_j - \frac{a_{sj} c_r}{a_{sr}}; \quad Z' = Z - \frac{c_r b_s}{a_{sr}};$$
$$i \in [1, m]; j \in [1, n];$$

9) Repeat the algorithm, starting from step 4, as long as there are non-negative elements in the row of objective function coefficients.

The solution is found in the first and last columns,

and the value of the objective function is in

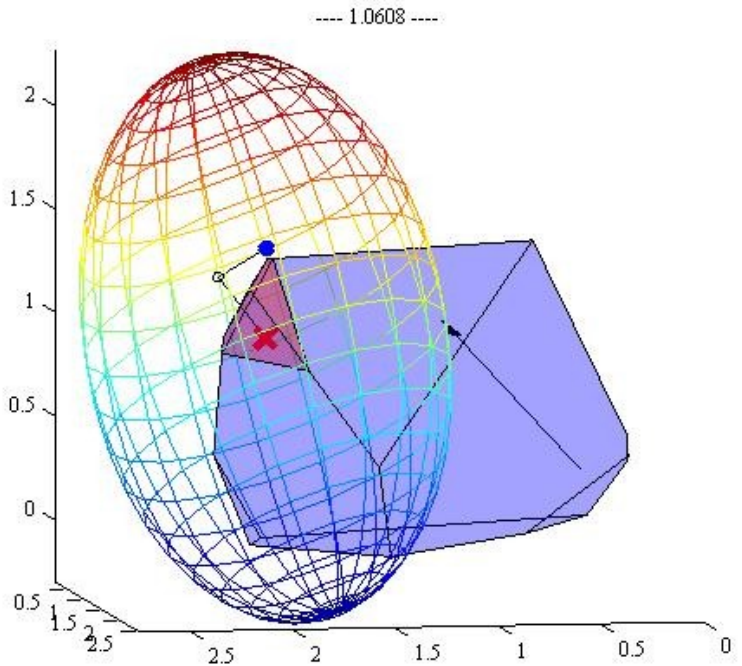
the bottom-right cell

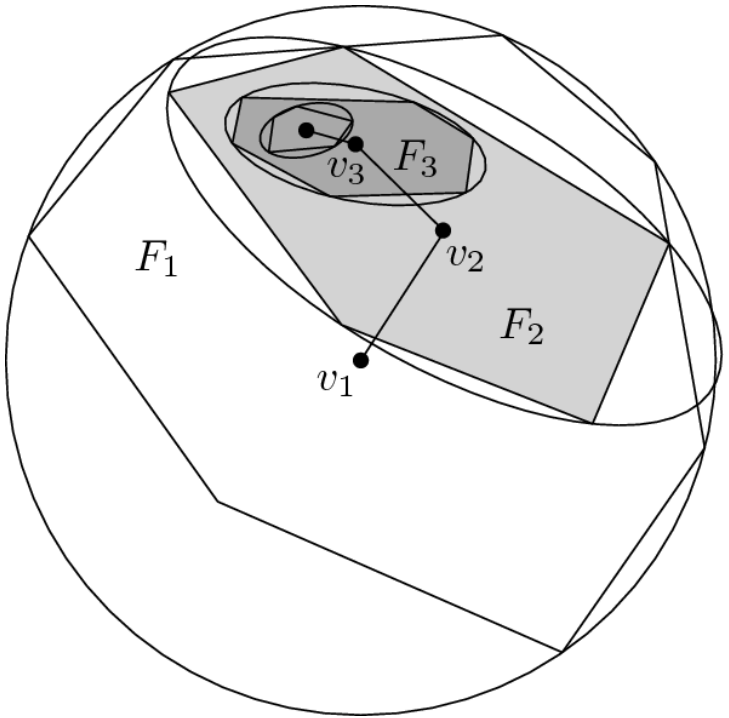
(with a minus sign).

basis	**variables**							$\mathbf{b_i}$
	x_1	x_2	...	x_n	y_1	...	y_m	
$\mathbf{x_2}$	0	1		0				$\mathbf{b'_1}$
$\mathbf{x_n}$	0	0		1				$\mathbf{b'_2}$
...								**...**
$\mathbf{x_1}$	1	0		0				$\mathbf{b'_m}$
$\mathbf{c_j}$	**<=0**	**<=0**	**…**	**<=0**	**<=0**	**…**	**<=0**	**– L**

Ellipsoid method

The **ellipsoid method** is an iterative method for minimizing convex functions over convex sets. The ellipsoid method generates a sequence of ellipsoids whose volume uniformly decreases at every step, thus enclosing a minimizer of a convex function.
A multidimensional generalization of the bisection method.





Ellipsoid method

If it is possible to construct a ball containing the desired solution in a linear programming problem, it can be solved using the ellipsoid method.

First find some point u inside the ball that satisfies the constraints of the problem.

Draw a hyperplane f (x) < f (u) through it, where f is the objective function, and find a point at the intersection of the original and new hyperplanes (starting with the current ellipsoid). the construction of the "separating hyperplane" is based on the inequality for convex functions

$$f(y) \geqslant f(x) + \nabla f(x)^T (y - x)$$

If we fix x, then for a convex function f the half-space $\nabla f(x)^T (y-x)$ contains only points with a value no less than at x.

After drawing the hyperplane, an ellipsoid of minimum volume is constructed, which contains one of the parts of the original. This can be achieved by adding new points. An ellipsoid can always be defined by a positive definite matrix and a vector (the center of the ellipsoid)

$$\mathrm{E}(P, x) = \{ z \mid (z - x)^T P (z - x) \leqslant 1 \}$$

Do the same with the newly found point.

The process converges to the optimal solution at an exponential rate (since the volume of the ellipsoid decreases at this rate).

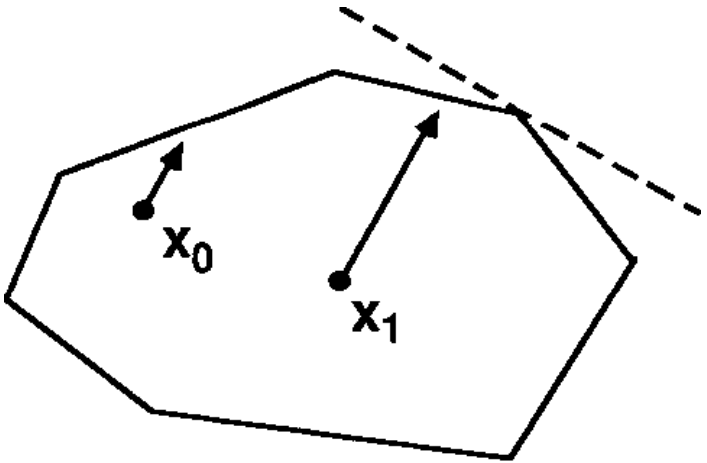
> There are problems with a size of 50 variables, which require more than 24 thousand iterations of the Khachiyan method, while the number of significantly simpler iterations of the simplex method in such cases is in the hundreds or even tens

Karmarkar's algorithm

I**nterior point method** - the current feasible solution does not move along the boundary of the feasible solution area as in the simplex method, but moves along the interior points of the feasible value area, improving with each iteration the approximation of the optimal solution by a certain fraction and leading to an optimal solution with rational data.

The basic idea of the method is to replace restrictions with a penalty in the form of a so-called barrier function.

To avoid colliding with the boundaries, the algorithm employs a clever mathematical trick—a projective transformation. It "inflates" the space around the current point, positioning it precisely at the center of the transformed shape, while pushing all the boundary constraints to an equal distance away.



$$F(x) \to \infty \text{ when } x \to \partial K; \quad \partial K \text{ is the boundary of the feasible area } K$$

$$\text{minimize } \phi(x) = tf(x) + F(x)$$

$$K: g_i(x) \leqslant 0, 1 \leqslant i \leqslant m \Rightarrow F(x) = -\sum_{i=1}^{m} \ln(-g(x))$$

$\hat{x} = argmin(\phi(x))$ for $t \in [0, \infty]$ draws the central path which leads to optimal solution

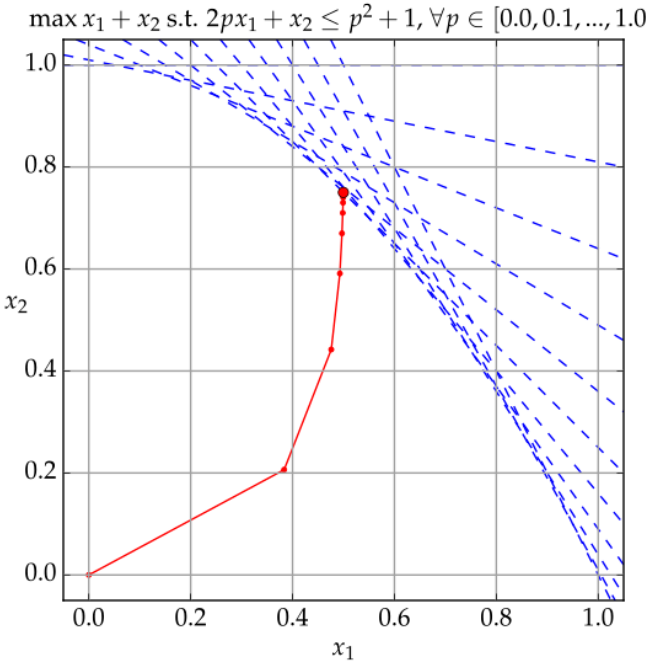
The step of Newton's method with the correct choice of the barrier function helps to generate a new point.

Karmarkar's algorithm

Algorithm

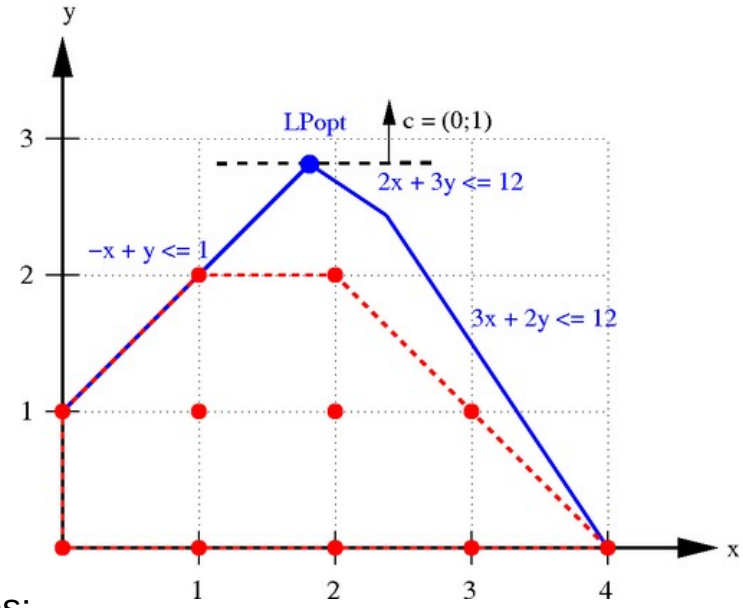
Problem: maximize $c^T x$; subject to $Ax \leqslant b$
Input: $A, b, c, x^0, stopping\ criterion, \gamma$
$k = 0$
while *stopping criteria not satisfied*
$v^k = b - Ax^k$
$D_v = diag(v_1^k \ldots v_m^k)$
$h_x = (A^T D^{-2} A)^{-1} c$
$h_v = -Ah_x$
if $h_v \geqslant 0$: *unbounded* (*terminates the algorithm*)
$\alpha = \gamma \min \{ \frac{-v_i^k}{(h_v)_i} | (h_v)_i < 0, i = 1 \ldots m \}$
$x^{k+1} = x^k + \alpha h_x$
$k = k + 1$

Example:
maximize $x_1 + x_2$
subject to $2p x_1 + x_2 \leq p^2 + 1$,
$p = 0.0, 0.1, 0.2, \ldots, 0.9, 1.0$.



Integer programming

Problem: same as for limear programming, but A, b, x must be integer.



Heuristic methods

- Hill climbing
- Simulated annealing
- Reactive search optimization
- Ant colony optimization
- Hopfield neural networks

Exact algorithms

Cutting plane methods, which work by solving the LP relaxation and then adding linear constraints that drive the solution towards being integer without excluding any integer feasible points.

Branch and bound method, which operates according to two principles:

It recursively splits the search space into smaller spaces, then minimizing f(x) on these smaller spaces; the splitting is called branching.

Branching alone would amount to brute-force enumeration of candidate solutions and testing them all. To improve on the performance of brute-force search, a B&B algorithm keeps track of bounds on the minimum that it is trying to find, and uses these bounds to "prune" the search space, eliminating candidate solutions that it can prove will not contain an optimal solution.

Integer programming

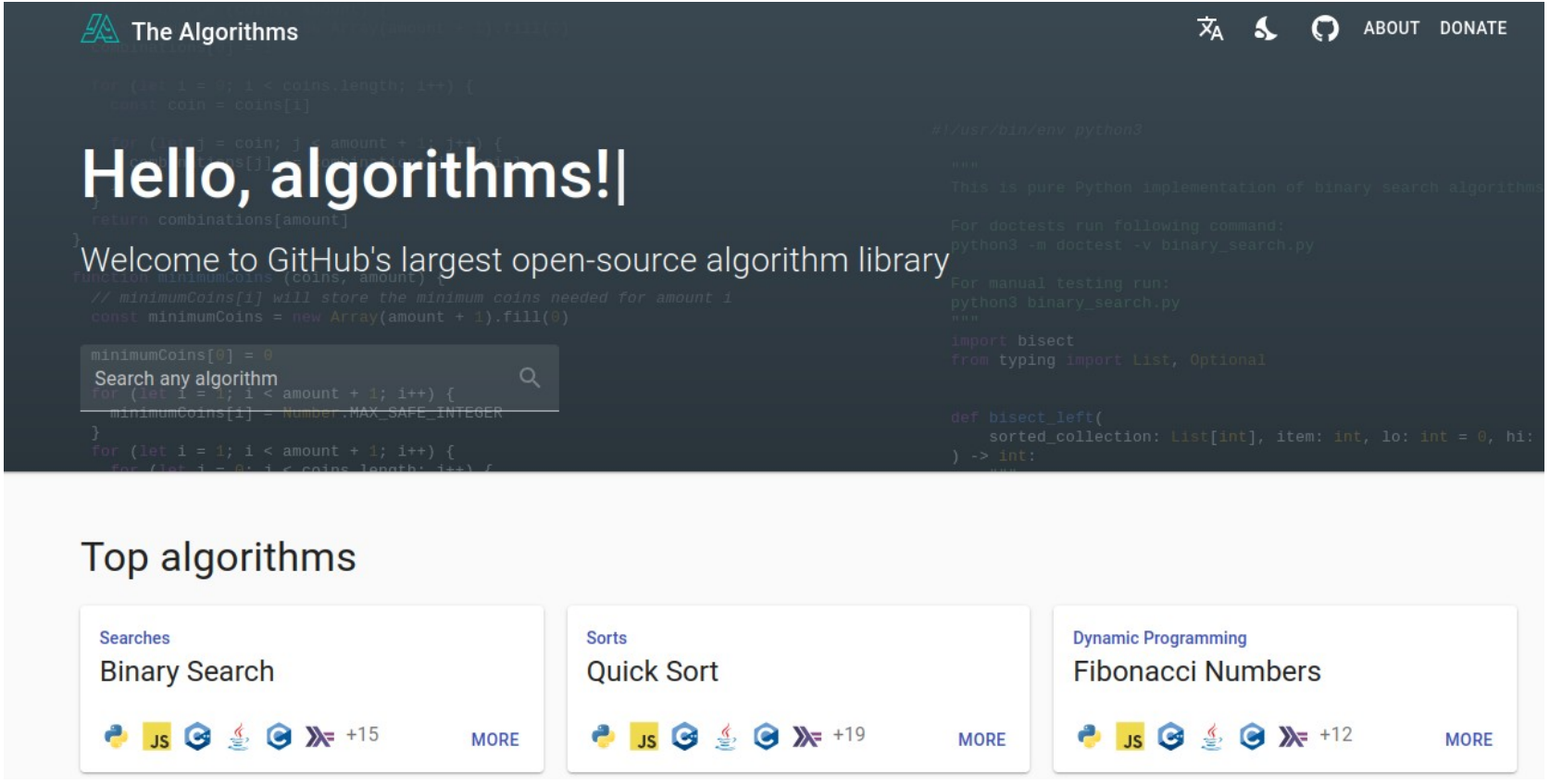
Algorithm:
1) Using a heuristic, find a solution xh to the optimization problem. Store its value, B = f(xh). (If no heuristic is available, set B to infinity.) B will denote the best solution found so far, and will be used as an upper bound on candidate solutions.
2) Initialize a queue to hold a partial solution with none of the variables of the problem assigned.
3) **Loop until** the queue is empty:
    Take a node N off the queue.
    If N represents a single candidate solution x and f(x) < B,
    then x is the best solution so far.
        Record it and set B ← f(x).
    else, branch on N to produce new nodes N_i.
    For each of these:
        If bound(N_i) > B,
        then do nothing; since the lower bound on this node is greater than the upper bound of the problem,
            It will never lead to the optimal solution, and can be discarded.
        else, store Ni on the queue.

FIFO queue-based implementation yields a breadth-first search.
A stack (LIFO queue) will yield a depth-first algorithm.

Examples of best-first search algorithms with this a priority queue that sorts nodes on their lower bound are Dijkstra's algorithm and A* search.

Collection

https://the-algorithms.com/





Антон Кованцев
pop.antonij@gmail.com
https://t.me/pop_antonij